Relational Model & SQL Basics

keys, constraints, join types, and why SQL

Watch the 4-min video

In one line: relational databases store data in typed tables linked by keys, protect it with constraints and ACID transactions, and answer questions you didn't plan for with joins, which makes them the right default for most business data.

1 Why SQL is still the default

An online shop has **customers**, **orders** and **order items**. Finance asks: *what was revenue per product last month?* Product asks: *which customers have never ordered?* Both are **one SQL query** away, if the data lives in a relational database. Nobody had to predict those questions when the schema was designed.

2 Tables, rows, keys

The relational model stores data in **tables**. Each **row** is one record; each **column** is one attribute with a **type**.

- A **primary key** (PK) uniquely identifies a row, like the order's `id`.
- A **foreign key** (FK) points to a row in another table, like `order_items.order_id` referencing `orders.id`. The database **enforces that link**: you can't add an item for an order that doesn't exist.

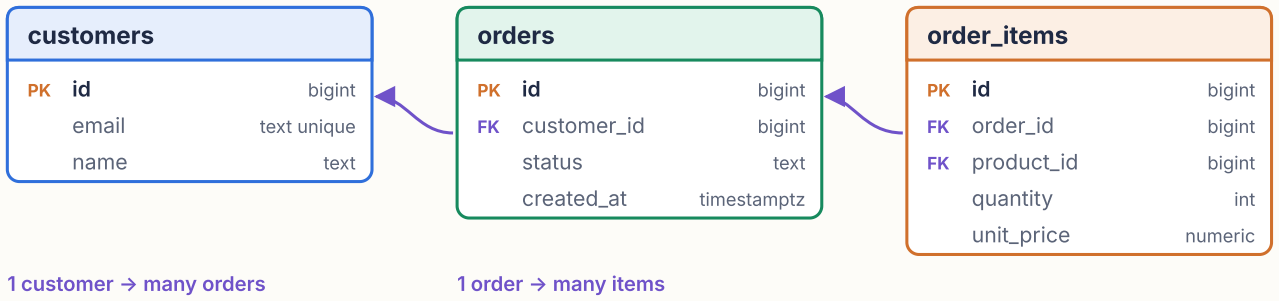


Figure 1. The shop schema. Each table holds one kind of thing; foreign keys connect them.

3 Constraints protect data at the source

NOT NULL, **UNIQUE**, **CHECK** (like `quantity > 0`) and **foreign keys** hold no matter which service, script or engineer writes to the table. Bugs in application code can't sneak bad data past them; the write simply fails.

```
CREATE TABLE orders (
  id          bigint PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers(id),
  status      text   NOT NULL CHECK (status IN ('pending','paid','cancelled')),
  created_at  timestamptz NOT NULL DEFAULT now()
);
-- order_items adds CHECK (quantity > 0) the same way

INSERT INTO orders (id, customer_id, status) VALUES (501, 999, 'paid');
-- ERROR: insert or update on table "orders" violates foreign key constraint
```

4 Joins combine tables

- **Inner join**: only matching rows. Orders that have customers.
- **Left join**: every row from the left table, with `NULL`s where there is no match. All customers, with their orders if they have any.
- **Anti join**: left rows with **no match at all**. Customers who never ordered, written with `NOT EXISTS` or a left join filtered on `IS NULL`.

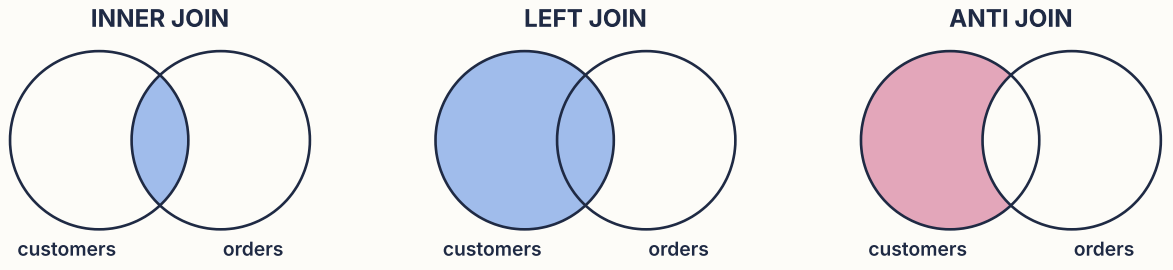


Figure 2. Shaded area = rows returned. An anti join is the left circle minus everything that overlaps.

```
-- customers who never ordered (anti join)
SELECT c.id, c.email
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- or: LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL;
```

5 Example: revenue per product last month

Join order items to orders, keep **paid** orders from **last month**, group by product, and sum quantity × unit price, sorted by revenue:

```
SELECT oi.product_id,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'paid'
  AND o.created_at >= date_trunc('month', now()) - interval '1 month'
  AND o.created_at <  date_trunc('month', now())
GROUP BY oi.product_id
ORDER BY revenue DESC;
```

It is one **declarative** query: you say *what* you want, and the **query optimizer** decides *how*, choosing join order and using indexes (for example one on `orders(status, created_at)`).

6 ACID transactions

Placing an order inserts the order, inserts its items and decrements stock, **all or nothing**. If any step fails, everything rolls back. No half-written orders.

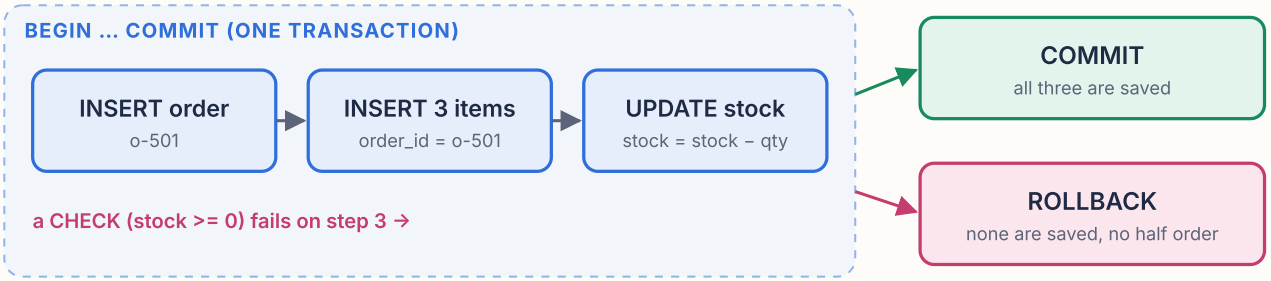


Figure 3. Atomicity in practice. Consistency comes from the constraints, isolation keeps other transactions from seeing the half-done order, and durability means a commit survives a crash.

WORKED EXAMPLE: WHEN IS RELATIONAL THE RIGHT DEFAULT?

When data is **structured and related**, when you need **transactions and constraints**, and when you'll ask questions you can't predict yet: ad hoc queries, reports and joins. **Postgres and MySQL** handle large datasets (hundreds of GB to a few TB) and **thousands of transactions per second on one node**, with read replicas for more reads. A shop with 1 million orders a month averages under 1 order per second, far inside that.

Reach for **NoSQL** when you have a specific need: massive write scale, very flexible schemas, or simple key-value access at huge volume.

COMMON MISTAKES

- Enforcing rules only in application code; the next script or service skips them.
- Using `NOT IN (subquery)` for an anti join: if the subquery returns any `NULL`, it returns no rows at all. Prefer `NOT EXISTS`.
- Putting a filter on the right table in `WHERE` after a left join, which silently turns it into an inner join (put it in `ON`).
- Choosing NoSQL "for scale" before you have a scale problem, then rebuilding joins and constraints by hand.

7 Key terms

Table / row / column: record set / one record / typed attribute

Constraint: rule the database enforces on every write

Anti join: left rows with no match

Primary key: uniquely identifies a row

Inner join: only matching rows

Query optimizer: picks how to execute a query

Foreign key: references a row in another table

Left join: all left rows, NULLs if no match

ACID: atomic, consistent, isolated, durable

8 Interview questions

Q1. When is a relational database the right default? BASIC

For structured, related data that needs transactions, constraints and flexible querying, which describes most business data. Choose something else only for a specific need: massive write scale, very flexible schemas, or simple key-value access at huge volume.

Q2. Explain the join types. BASIC

An inner join returns only matching rows. A left join keeps all rows from the left side, with NULLs where nothing matches. An anti join returns left rows with no match, like customers without orders, using NOT EXISTS or a left join filtered on IS NULL.

Q3. What is the difference between a primary key and a foreign key? BASIC

A primary key uniquely identifies a row in its own table (orders.id). A foreign key is a column that must match a primary key in another table (order_items.order_id → orders.id), so orphan rows are rejected.

Q4. Write a query for customers who never placed an order. SCENARIO

SELECT c.* FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id). Or LEFT JOIN orders and keep rows WHERE o.id IS NULL. An index on orders(customer_id) keeps it fast.

Q5. A bug in one service wrote order items with quantity 0 and −3. How do you stop it for good? SCENARIO

Add CHECK (quantity > 0) on the column (after cleaning existing bad rows). Then every writer, including scripts and other services, gets an error instead of corrupting data.

Q6. Placing an order needs 3 writes. The server crashes after the second. What happens? SCENARIO

Inside one transaction, nothing is committed, so the database rolls back both writes on restart: no order without stock change. Without a transaction you'd have an order with items but stock never decremented.

Q7. A LEFT JOIN query returns fewer customers than expected. Why? TRICKY

A condition on the right table in WHERE (e.g. WHERE o.status = 'paid') removes the NULL rows, turning it into an inner join. Move the condition into the ON clause.

Q8. Why can NOT IN return zero rows when you expect many? TRICKY

If the subquery returns a NULL, “x NOT IN (..., NULL)” is never true, it is unknown, so no row passes. NOT EXISTS doesn't have this problem.

Next: SD-E-002 · Database Normalization backendroadmap.com